

INFORME DE PRUEBAS UNITARIAS

Práctica 5B: Prueba de Estructuras de Datos

Asignatura: Ingeniería del Software II

Curso: 3º Grado en Ingeniería Informática

Universidad: Universidad de Cantabria

Autores: Santiago Jerez · Sebastián Soriano

Fecha: 5 de mayo de 2026

1. Introducción

En esta práctica hemos llevado a cabo el proceso completo de pruebas unitarias sobre la clase `ConjuntoOrdenado<E>`, que implementa la interfaz `IConjuntoOrdenado<E>`. Para ello hemos aplicado técnicas de caja negra (partición equivalente y análisis de valores límite) y caja blanca (cobertura de sentencias y decisión/condición), utilizando JUnit 5 como framework de pruebas y JaCoCo para medir la cobertura alcanzada.

El objetivo principal ha sido verificar el correcto funcionamiento de todos los métodos de la clase, detectar posibles defectos en la implementación y garantizar una cobertura del código del 100% tras su corrección.

2. Pruebas de Caja Negra

Para el diseño de los casos de prueba de caja negra nos hemos basado exclusivamente en la documentación de la interfaz `IConjuntoOrdenado`, aplicando las técnicas de partición equivalente y análisis de valores límite (AVL).

2.1 Método `add(E elemento)`

ID	Descripción	Entrada	Resultado esperado
CN-A1	Elemento nulo	<code>add(null)</code>	<code>NullPointerException</code>
CN-A2	Añadir en lista vacía	<code>add(5)</code>	<code>true</code> , <code>size=1</code>
CN-A3	Orden ascendente	<code>add(5)</code> , <code>add(3)</code> , <code>add(7)</code>	Lista: <code>[3, 5, 7]</code>
CN-A4	Elemento duplicado	<code>add(5)</code> dos veces	<code>false</code> , <code>size=1</code>

```
@Test
public void testAddMantienOrdenAscendente() {
    conjunto.add(5);
    conjunto.add(3);
    conjunto.add(7);
    // Orden esperado: [3, 5, 7]
    assertEquals(3, conjunto.get(0));
    assertEquals(5, conjunto.get(1));
    assertEquals(7, conjunto.get(2));
}

@Test
public void testAddDuplicadoRetornaFalse() {
    conjunto.add(5);
    assertFalse(conjunto.add(5));
    assertEquals(1, conjunto.size());
}
```

2.2 Método `get(int index)`

ID	Descripción	Entrada	Resultado esperado
CN-G1	Índice válido	<code>get(0)</code> con 1 elemento	Devuelve el elemento
CN-G2	Índice negativo (AVL)	<code>get(-1)</code>	<code>IndexOutOfBoundsException</code>
CN-G3	Índice igual a size (AVL)	<code>get(5)</code> con 2 elementos	<code>IndexOutOfBoundsException</code>

2.3 Método remove(int index)

ID	Descripción	Entrada	Resultado esperado
CN-R1	Índice válido	remove(0) con 1 elemento	Devuelve elemento, size=0
CN-R2	Índice inválido (AVL)	remove(-1)	IndexOutOfBoundsException

2.4 Métodos size() y clear()

ID	Descripción	Entrada	Resultado esperado
CN-S1	Size en lista vacía	size()	0
CN-SC1	Clear vacía la lista	add(1), add(2), clear()	size=0

3. Defectos Encontrados

Al ejecutar los tests de caja negra sobre la implementación original, detectamos 2 defectos en el método `add()`. A continuación se describe cada uno de ellos, el test que lo evidenció y la corrección aplicada.

3.1 Bug 1 — Orden de inserción incorrecto

Test que lo detectó: `testAddMantienOrdenAscendente`

Síntoma: Al insertar los elementos [5, 3, 7], la lista resultante era [7, 5, 3] en vez de [3, 5, 7]. La posición 0 devolvía 7 cuando se esperaba 3.

Causa y corrección:

La condición del bucle `while` usaba el operador `< 0` en lugar de `> 0`, lo que provocaba que el índice de inserción se calculara en sentido inverso al deseado.

```
[INFO] Running es.unican.is2.ConjuntoOrdenadoTest
[ERROR] Tests run: 11, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0.064 s <<< FAILURE! -- in es.unican.is2.ConjuntoOrdenadoTest
[ERROR] es.unican.is2.ConjuntoOrdenadoTest.testAddMantienOrdenAscendente -- Time elapsed: 0.006 s <<< FAILURE!
org.opentest4j.AssertionFailedError: expected: <3> but was: <7>
    at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
    at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
    at org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
    at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:182)
    at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:177)
    at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:534)
    at es.unican.is2.ConjuntoOrdenadoTest.testAddMantienOrdenAscendente(ConjuntoOrdenadoTest.java:36)
    at java.base/java.lang.reflect.Method.invoke(Method.java:565)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
```

3.2 Bug 2 — No detecta elementos duplicados

Test que lo detectó: `testAddDuplicadoRetornaFalse`

Síntoma: Al intentar añadir un elemento ya existente, el método devolvía `true` e insertaba el duplicado en la lista, incumpliendo el contrato de la interfaz.

Causa y corrección:

La implementación original no incluía ninguna comprobación de duplicados. Se añadió una condición tras el `while` que verifica si el elemento en la posición calculada es igual al elemento a insertar, retornando `false` en ese caso.

```
public boolean add(E elemento) {
    int indice = 0;
    if (elemento==null)
        throw new NullPointerException();
    if (lista.size() != 0) {
        while (indice < lista.size() && elemento.compareTo(lista.get(indice)) > 0) {
            indice++;
        }
    }
    if (indice < lista.size() && elemento.compareTo(lista.get(indice))==0)
    {
        return false;
    }
    lista.add(indice, elemento);
    return true;
}
```

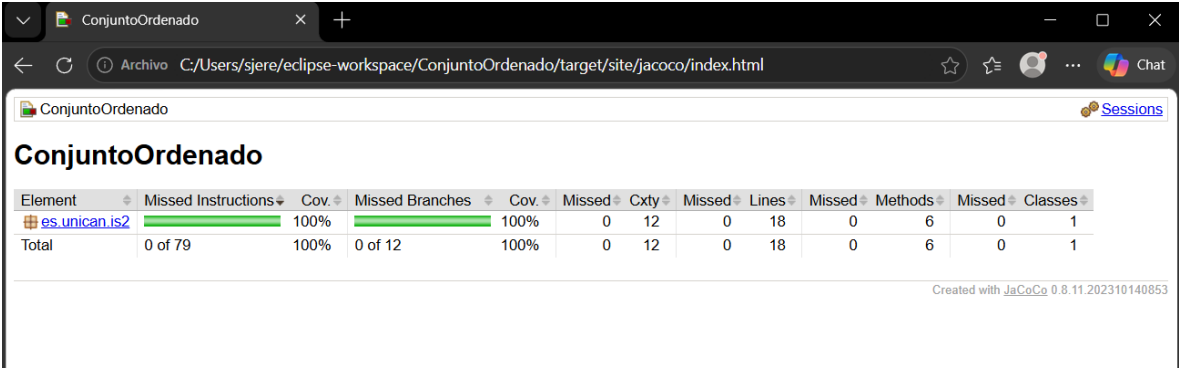
4. Pruebas de Caja Blanca — Cobertura

Tras corregir los defectos, ejecutamos el análisis de cobertura con el plugin JaCoCo 0.8.11 integrado en Maven. Los resultados obtenidos sobre la clase ConjuntoOrdenado fueron los siguientes:

Métrica	Total	Cubiertas	Cobertura
Instrucciones	79	79	100%
Ramas (Branches)	12	12	100%
Líneas	18	18	100%
Métodos	6	6	100%
Clases	1	1	100%

Los 11 casos de prueba diseñados en la fase de caja negra fueron suficientes para alcanzar cobertura total, por lo que no fue necesario añadir casos adicionales de caja blanca.

```
[INFO] Running es.unican.is2.ConjuntoOrdenadoTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.06
2 s -- in es.unican.is2.ConjuntoOrdenadoTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ ConjuntoOrdenado ---
[INFO] Loading execution data file C:\Users\sjere\eclipse-workspace\Conjunto
Ordenado\target\jacoco.exec
[INFO] Analyzed bundle 'ConjuntoOrdenado' with 1 classes
[INFO]
-----
[INFO] BUILD SUCCESS
[INFO]
-----
[INFO] Total time: 1.593 s
[INFO] Finished at: 2026-05-05T07:44:58-04:00
[INFO]
-----
---
```



5. Conclusión

El proceso de pruebas ha resultado satisfactorio. Los 11 casos de prueba diseñados mediante técnicas de caja negra (partición equivalente y AVL) detectaron correctamente los 2 defectos presentes en la implementación original del método `add()` de la clase `ConjuntoOrdenado`.

Tras la corrección de ambos defectos, todos los tests pasaron con éxito y el análisis de cobertura con JaCoCo confirmó que se alcanzó el 100% en todas las métricas (instrucciones, ramas, líneas, métodos y clases), cumpliendo con los requisitos de caja blanca establecidos en el enunciado de la práctica.